

DOKUMEN TUGAS BESAR

IF2224 Teori Bahasa Formal dan Automata

Arion Compiler : Milestone 1 - Lexical Analysis



Nama Anggota :

Zahran Alvan Putra Winarko	13524124
Neswa Eka Anggara	13524136
Daniel Putra Rywandi S.	13524143
Muh. Hartawan Haidir	13524147

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026

BAB I

TEORI SINGKAT

1. Data Flow Diagram (DFD)

Data Flow Diagram (DFD) adalah representasi visual atau penggambaran diagram dari aliran data mengenai suatu proses atau sistem informasi. DFD berfokus kepada proses yang mengolah data, sumber data, dan tujuan akhir dari aliran data tersebut, sehingga DFD tidak mempunyai kontrol terhadap flow yang menyebabkan hilangnya aturan atas keputusan atau pengulangan. DFD menggunakan simbol grafis untuk mengilustrasikan jalur, proses, dan tempat penyimpanan data dari titik awal data masuk ke dalam sistem sehingga data tersebut keluar. Model visual ini membantu para profesional mengidentifikasi cara-cara untuk meningkatkan efisiensi dan efektivitas sistem dan proses yang ada. Berikut adalah elemen-elemen yang ada pada DFD :

- **Entitas Eksternal (External Entity)**

External entity atau lebih sering disebut dengan terminator merupakan pihak di luar sistem, dapat berupa individu, divisi, perusahaan, atau sistem yang lainnya. Terminator dapat memberikan masukan atau keluaran terhadap sistem. Simbol dari external entity dilambangkan dengan persegi panjang atau kotak.

- **Proses (Process)**

Process dilakukan oleh mesin dengan mengubah input menjadi output dengan format yang berbeda. Simbol proses digambarkan dalam bentuk lingkaran, oval, atau persegi panjang dengan tambahan sudut bundar.

- **Aliran Data (Data Flow)**

Data flow merupakan arus data yang mengalir antara terminator, proses, dan data store. Data flow digambarkan dengan simbol tanda panah, dan fungsi utamanya adalah untuk mengalirkan informasi dari satu sistem ke sistem yang lain.

- **Penyimpanan Data (Data Store)**

Data store adalah file untuk menyimpan data yang digunakan untuk proses selanjutnya. Dapat dikatakan juga, sama seperti basis data (database). Pada umumnya, data store berupa tabel yang dapat diolah, serta mampu terhubung dengan setidaknya satu masukan dan satu keluaran. Penggambaran atau simbol data store berupa dua garis sejajar.

Data Flow Diagram berbeda dengan **UML (*Unified Modelling Language*)**, dimana hal mendasar yang menjadi pembeda antara kedua skema tersebut terletak pada flow dan objective penyampaian informasi di dalamnya. Terdapat tiga fungsi dari pembuatan diagram alir data untuk kebutuhan *Software development*. Berikut adalah fungsi beserta penjelasannya

1. **Menyampaikan rancangan sistem**

Dengan pembuatan DFD, maka proses penyampaian informasi menjadi lebih mudah dengan tampilan visual yang simple dan dapat dimengerti oleh tiap stakeholder. Dimana, data yang disajikan mampu menggambarkan alur data secara terstruktur dengan pendekatan yang lebih efisien.

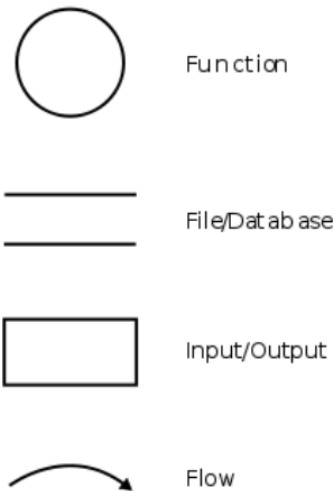
2. **Menggambarkan suatu sistem**

Fungsi yang kedua, DFD dapat membantu proses penggambaran sistem sebagai jaringan fungsional. Maksudnya adalah, di dalam jaringan terdapat berbagai komponen yang saling terhubung menggunakan alur data.

3. **Perancangan model**

Fungsi yang terakhir, diagram ini juga dapat membuat rancangan model baru dengan menekankan pada fungsi sistem tertentu. Hal tersebut dapat dimanfaatkan untuk melihat bagian yang lebih detail dari diagram alir data tersebut.

Terdapat beberapa simbol utama untuk menyusun sebuah rangkaian DFD yang tepat, diantaranya sebagai berikut :



Selain itu, Data flow juga terbagi menjadi tiga jenis, dimana setiap bagiannya memiliki peran dan fungsinya masing-masing. Untuk pembuatannya sendiri dapat menyesuaikan kebutuhan proyek dari manajemen tim. Berikut adalah jenisnya

1. Diagram Level 0 (Diagram Konteks)

Diagram konteks atau level 0 merupakan diagram dengan tingkatan paling rendah, dimana menggambarkan sistem berinteraksi dengan entitas eksternal. Pada diagram konteks akan diberi nomor untuk setiap proses yang berjalan, dimulai dari angka 0 terlebih dahulu. Jadi, untuk setiap aliran data akan langsung diarahkan menuju sistem. Ciri dari diagram level 0 terletak pada tidak adanya informasi yang terkait data yang tersimpan pada data store.

2. Diagram Level 1

DFD level 1 merupakan lanjutan dari diagram konteks karena setiap proses yang berjalan akan diperinci pada tingkatan ini sehingga proses utama akan dipecah menjadi sub-sub proses yang lebih kecil lagi.

3. Diagram Level 2

DFD level 2 merupakan tingkat lanjutan dari level yang sebelumnya, dimana pada fase ini akan dijelaskan lebih detail terkait tiap prosesnya. Namun, untuk tingkatan ini jarang sekali dikerjakan dan lebih banyak hanya menerapkan dua level di bawahnya saja.

Setelah mengetahui jenis pada DFD, selanjutnya kita akan masuk kepada cara untuk membuat data flow diagram yang baik dan benar. Berikut adalah langkah-langkah yang harus dilakukan

1. Data Store harus diproses

Pertama, yang perlu anda perhatikan adalah setiap data yang tersimpan di dalam data store harus diproses lebih lanjut untuk dijadikan sebagai keluaran (output).

2. Menentukan jumlah input dan output

Kedua, pada setiap DFD setidaknya mempunyai satu inputan dan satu keluaran karena diagram alir data harus mencerminkan alur sistem dari tahap awal hingga akhir.

3. Hubungan pada Data Store

Ketiga, setiap data store harus saling terhubung dengan setidaknya satu input dan satu output agar dapat menyimpan data yang masuk menuju sistem.

4. Letak posisi Proses

Aturan terakhir, setiap proses yang telah terjadi pada diagram alir data harus melalui proses untuk menghasilkan output yang sesuai.

2. Deterministic Finite Automata (DFA)

Finite State Automata adalah mesin abstrak yang memiliki lima elemen atau tuple. Kelima elemen tersebut meliputi input, output, himpunan, state, relasi state, dan relasi output. Dalam arti yang lebih jelas, Finite State Automata berupa sistem model matematika dengan masukan dan keluaran diskrit yang dapat mengenali bahasa paling sederhana (bahasa reguler) dan dapat diimplementasikan secara nyata. Finite State Automata dapat menerima input dan mengeluarkan

output yang memiliki state yang berhingga banyaknya. Selain itu, Finite State Automata memiliki sekumpulan aturan untuk berpindah dari satu state ke state yang lain berdasarkan input dan fungsi transisi yang diterapkan. Pada dasarnya, penggunaan Finite State Automata adalah untuk mengenali pola. Dibutuhkan string simbol sebagai input dan statusnya berubah sesuai dengan input tersebut. Ketika ditemukan simbol input yang diinginkan, maka terjadi transisi. Pada saat transisi, automata dapat berpindah ke keadaan berikutnya atau tetap dalam keadaan yang sama. Finite State Automata memiliki dua status, status **Terima** atau status **Tolak**. Ketika string input berhasil diproses, dan automata mencapai state akhir, maka statusnya adalah Terima, demikian juga sebaliknya.

Finite State Automata dapat didefinisikan dengan persamaan berikut :

$$M = (Q, \Sigma, \delta, S, F)$$

- Q = himpunan state
- Σ = himpunan simbol input
- δ = fungsi transisi $\delta : Q \times \Sigma$
- S = state awal / initial state , $S \in Q$
- F = state akhir, $F \subseteq Q$

Finite State Automata memiliki beberapa karakteristik berikut :

- Setiap Finite Automata memiliki keadaan dan transisi yang terbatas.
- Transisi dari satu keadaan ke keadaan lainnya dapat bersifat deterministik atau non-deterministik.
- Setiap Finite Automata selalu memiliki keadaan awal.
- Finite Automata dapat memiliki lebih dari satu keadaan akhir.
- Jika setelah pemrosesan seluruh string, keadaan akhir dicapai, artinya automata menerima string tersebut

State Finite Automata terbagi menjadi dua jenis, yaitu :

1. Deterministic Finite Automata (DFA)

2. Non-Deterministic Finite Automata (NFA)

Pada Milestone 1 ini, penggunaan State Finite Automata akan lebih terfokus kepada Deterministic Finite Automata (DFA) yang dimana mesin hanya dapat menuju satu state dan fungsi transisi dipakai pada setiap state untuk setiap simbol input. Oleh karena itu, implementasi DFA akan lebih sulit dibanding NFA, tetapi DFA akan menuntun recognizer lebih cepat dibanding NFA. Deterministic Finite Automata terdiri dari 5 tuple, yakni :

$$\{Q, \Sigma, q, F, \delta\}$$

- **Q** : himpunan semua state
- **Σ** : himpunan simbol input (simbol yang digunakan oleh mesin untuk mengambil nilai input)
- **q** : initial state (state atau keadaan awal dari mesin)
- **F** : himpunan state akhir
- **δ** : fungsi transisi, yang didefinisikan $\delta : Q \times \Sigma \rightarrow Q$.

Satu hal penting yang perlu diperhatikan pada jenis DFA adalah kemungkinan pada sebuah pola yang terbentuk akan sangat banyak dibanding NFA. Namun secara umum, DFA dengan jumlah state minimum cenderung lebih baik. Oleh karena itu, penggunaan DFA dan NFA harus disesuaikan dengan jenis kasus yang ada.

3. Lexical Analyzer (lexer)

Analisis leksikal, juga dikenal sebagai pemindaian, adalah fase pertama dari sebuah kompilator. Pada fase ini, kompilator membaca kode sumber karakter demi karakter dari kiri ke kanan dan mengelompokkannya menjadi unit-unit bermakna yang disebut token. Token ini kemudian diteruskan ke fase kompilasi berikutnya, yang dikenal sebagai analisis sintaksis.

Dalam bahasa pemrograman, token didefinisikan menggunakan ekspresi reguler (**penjelasan token ada dibawah**). Penganalisis leksikal (pemindai) menggunakan otomaton hingga Deterministik (DFA) untuk mengenali token-token ini karena DFA dapat mengidentifikasi bahasa reguler secara efisien. Setiap keadaan akhir DFA mewakili tipe token tertentu. Proses konversi ekspresi reguler menjadi DFA dapat diotomatisasi, sehingga pengenalan token menjadi cepat dan sistematis. Untuk lebih mengetahui tentang penganalisa leksikal, berikut adalah arsitektur yang digunakan untuk melakukan pemindaian :

1. Membaca Kode sumber

Penganalisis leksikal memindai seluruh kode sumber dan mengidentifikasi berbagai komponen seperti kata kunci, operator, variabel, dan simbol

2. Pembuatan Token

Setiap komponen yang teridentifikasi dikonversi menjadi token (unit kode yang bermakna). Sebagai contoh adalah `int a = 5;`, token yang dihasilkan adalah

- `Int` → kata kunci
- `A` → pengidentifikasi
- `=` → operator penugasan
- `5` → Konstanta
- `;` → Simbol khusus

3. Mengabaikan elemen tambahan

Penganalisis melewati spasi dan menghapus komentar karena tidak diperlukan untuk eksekusi

4. Penanganan kesalahan

Jika ditemukan karakter yang tidak valid atau simbol yang tidak dikenal, penganalisis akan melaporkan kesalahan beserta nomor baris dalam file sumber

5. Interaksi dengan parser

Parser meminta token dari penganalisis leksikal sesuai kebutuhan. Penganalisis leksikal tidak menghasilkan semua token sekaligus, tetapi menyediakannya berdasarkan permintaan.

```
int main) (  
}  
x = y + z;  
int x, y, z;  
print("Goto GFG %d%d", a);  
{
```

Pada fase pertama, kompiler tidak memeriksa sintaks. Jadi, program ini dimasukkan sebagai input ke penganalisis leksikal dan diubah menjadi token. Oleh karena itu, tokenisasi adalah salah satu fungsi penting dari penganalisis leksikal. Jumlah total token untuk program ini adalah 26. Diagram di bawah ini menunjukkan bagaimana token akan dihitung.

```
|int | main |) | (
| } |
|x| = |y| + |z| ;| |
|int| x| ,| y|,| z|;|
|print| ( | "Goto GFG %d%d" |,| a| ) |;|
|{|
```

----- Will count as 1

Total Token = 26

Pada diagram diatas, anda dapat memeriksa dan menghitung jumlah token dan memahami bagaimana tokenisasi bekerja pada fase penganalisis leksikal. Dengan cara ini, anda dapat memahami setiap fase dalam kompiler dengan jelas dan mendapatkan gambaran tentang bagaimana compiler bekerja secara internal dan setiap fase kompiler merupakan langkah kunci.

Selain arsitektur yang digunakan, akan ditunjukkan bagaimana penganalisa leksikal bekerja :

1. Pra-pemrosesan input

Tahap ini melibatkan pembersihan input dan persiapan analisis leksikal, yang mungkin termasuk menghapus komentar, spasi kosong, dan teks non-input lainnya dari teks input.

2. Tokenisasi

Ini adalah proses memecah teks masukan menjadi serangkaian token.

3. Klasifikasi token

Leksik menentukan jenis setiap token, yang dapat diklasifikasikan sebagai kata kunci, pengenal, angka, operator, dan pemisah.

4. Validasi token

Lexeme memeriksa setiap token apakah valid sesuai dengan aturan bahasa pemrograman.

5. Generasi Keluaran

Ini adalah tahap akhir dari proses analisis leksikal, di mana leksim menghasilkan keluaran berupa daftar token.

Ketika sudah mengetahui tentang cara kerja dari penganalisis leksikal, maka kita juga harus mengetahui peran yang dijalankan leksikal terhadap suatu pemindaian, yaitu :

- **Bersihkan kode sumbernya**

Kode sumber seringkali berisi karakter tambahan yang tidak dibutuhkan oleh kompiler atau interpreter, seperti spasi, tab, baris baru, dan komentar. Penganalisis leksikal menghapus karakter-karakter ini untuk mempermudah tahap pemrosesan selanjutnya. Bayangkan seperti membersihkan meja Anda sebelum memulai sebuah proyek.

- **Catat kesalahan**

Saat melakukan pembersihan, penganalisis leksikal mungkin menemukan karakter yang tidak valid atau rangkaian karakter yang tidak membentuk token yang valid. Misalnya, ia mungkin menemukan karakter yang tidak diizinkan dalam bahasa pemrograman. Ketika ini terjadi, ia melaporkan pesan kesalahan dan idealnya menunjukkan lokasi kesalahan dalam kode sumber asli (misalnya, nomor baris dan posisi karakter). Ini membantu programmer menemukan dan memperbaiki kesalahan.

- **Cari tahu komponen dasar (token) yang dibutuhkan**

Inilah tugas intinya. Penganalisis leksikal memindai kode sumber yang telah dibersihkan dan mengelompokkan karakter menjadi unit-unit bermakna yang disebut token . Token-token ini adalah blok bangunan fundamental dari program. Contoh token meliputi: Penganalisis leksikal tidak memahami arti dari token-token ini; ia hanya mengidentifikasi apa token tersebut.

1. Kata kunci
2. Pengidentifikasi
3. Operator
4. Literal
5. Tanda Baca

- **Baca kode sumber karakter demi karakter**

Penganalisis leksikal membaca kode sumber satu karakter demi satu karakter, mengelompokkan karakter-karakter ini menjadi token. Ini seperti membaca kalimat kata demi kata, tetapi pada tingkat yang lebih mendasar – karakter demi karakter untuk membentuk kata-kata (token). Pembacaan karakter demi karakter ini memungkinkan penganalisis untuk mengenali bahkan token yang kompleks.

4. Token

Token merupakan unit terkecil yang memiliki makna dalam suatu bahasa pemrograman. Dalam proses lexical analysis, token dihasilkan dari pengelompokkan karakter-karakter dalam source code yang membentuk suatu pola tertentu. Setiap token umumnya terdiri dari dua komponen utama, yaitu jenis token (token type) dan nilai token (token value).

Sebagai contoh, pada potongan kode :

`a := 5;`

Akan dihasilkan token :

- `ident(a)` → identifier
- `Becomes` → operator assignment
- `intcon(5)` → konstanta integer
- `Semicolon` → simbol akhir pernyataan

Dalam spesifikasi bahasa arion, terdapat berbagai jenis token yang harus dikenali oleh lexer, antara lain :

- Keyword : seperti `program`, `var`, `begin`, `end`
- Identifier (ident): nama variabel, fungsi, atau konstanta
- Operator : seperti `+`, `-`, `*`, `/`, `div`, `mod`
- Literal / Konstanta : seperti integer (intcon), real (realcon), string, dan karakter
- Simbol : seperti `;`, `,`, `(`, `)`, `:`
- Comment : teks yang diabaikan oleh lexer

Token sangat penting karena menjadi input bagi tahap selanjutnya, yaitu syntax analysis. Dengan adanya tokenisasi, proses parsing menjadi lebih terstruktur karena tidak lagi berurusan dengan karakter mentah, melainkan dengan unit yang sudah memiliki makna

5. Arion Compiler

Arion Compiler merupakan sistem yang dikembangkan dalam tugas besar ini untuk mengolah bahasa pemrograman Arion, yaitu bahasa pemrograman sederhana yang dirancang khusus untuk keperluan pembelajaran konsep compiler dan automata. Secara umum, compiler atau interpreter memiliki beberapa tahapan utama dalam memproses source code, yaitu :

1. **Lexical Analysis**
2. **Syntax Analysis**
3. **Semantic Analysis**
4. **Intermediate Code Generation**
5. **Execution**

Pada milestone pertama ini, fokus utama adalah pada tahap Lexical Analysis, yaitu proses mengubah source code menjadi token. Tahap ini merupakan pondasi penting karena hasil dari lexer akan digunakan oleh tahap-tahap selanjutnya. Bahasa Arion sendiri memiliki aturan token yang telah ditentukan, seperti daftar keyword, operator, dan struktur penulisan identifier. Lexer yang dibangun harus mampu mengenali seluruh aturan tersebut dan menghasilkan output berupa daftar token sesuai format yang ditentukan.

Selain itu, Arion Compiler dalam tugas ini diimplementasikan menggunakan bahasa C/C++ dan wajib mengikuti pendekatan **Deterministic Finite Automata (DFA)** dalam proses analisis leksikal. Hal ini bertujuan agar mahasiswa memahami bagaimana automata digunakan secara nyata dalam pengolahan bahasa formal. Dengan demikian, Arion Compiler tidak hanya berfungsi



sebagai alat pemroses bahasa, tetapi juga sebagai media pembelajaran untuk memahami konsep dasar compiler, khususnya pada tahap lexical analysis.

BAB II

PERANCANGAN & IMPLEMENTASI

1. DFA

1. Penjelasan Umum

DFA ini dirancang untuk membaca karakter dari source code secara sekuensial dan menghasilkan token yang valid untuk bahasa Arion. State Q0 bertindak sebagai start state di mana proses pembacaan karakter dimulai. Pada state ini, terdapat looping untuk karakter spasi, tab, dan newline yang mengindikasikan bahwa karakter-karakter whitespace tersebut akan diabaikan oleh lexer.

2. Penanganan Identifier

Transisi: Q0 -> (a-z, A-Z) -> Q1

Ketika lexer menemukan karakter alfabet, transisi bergerak ke Q1. State Q1 memiliki looping untuk menerima kombinasi huruf dan angka (a-z, A-Z, 0-9).

3. Penanganan Numerik

Transisi: Q0 -> (0-9) -> Q2

Angka pertama yang dibaca akan membawa lexer ke Q2 yang memiliki looping untuk angka (0-9). State Q2 merupakan final state untuk token intcon

4. Penanganan Karakter dan String

Transisi: Q0 -> (') -> Q5 -> (semua karakter kecuali ') -> Q5 -> (') -> Q6

Tanda petik tunggal menginisiasi pembacaan teks. State Q5 akan terus membaca karakter apapun selain tanda petik penutup. Ketika tanda petik penutup (') ditemukan, transisi berakhir di Q6.

5. Penanganan Komentar

Komentar kurung kurawal : Menggunakan jalur $Q0 \rightarrow Q7 \rightarrow Q8$. State $Q7$ akan membaca dan membuang semua karakter selain `}`. Ketika `}` ditemukan, proses pembacaan komentar selesai di state $Q8$. Komentar kurung bintang : Menggunakan jalur $Q0 \rightarrow Q9 \rightarrow Q10 \rightarrow Q11 \rightarrow Q12$. Transisi diawali dengan pembacaan `(` lalu `*`. State $Q10$ dan $Q11$ membentuk sirkulasi untuk mengabaikan karakter di dalam komentar sambil mencari `*` dan `)`. Transisi akan bergeser ke $Q12$ dan mengakhiri komentar hanya jika karakter `*` langsung diikuti oleh `)`

6. Penanganan Operator Relasional

Penanganan operator relasional dan logika pada diagram DFA ini dirancang agar mampu mengenali baik operator tunggal maupun operator majemuk melalui mekanisme percabangan antar state. Saat lexer membaca simbol kurang dari (`<`), proses akan berpindah ke state $Q15$ yang secara langsung mengidentifikasi token. Dari state tersebut, terdapat dua kemungkinan transisi lanjutan. Jika karakter berikutnya adalah `>`, maka lexer berpindah ke state $Q17$ untuk menghasilkan token (`<>`). Sementara itu, jika karakter berikutnya adalah `=`, maka transisi dilanjutkan ke state $Q16$ guna mengenali token (`<=`). Pendekatan yang sama juga diterapkan pada simbol lebih dari (`>`), yang pertama kali masuk ke state $Q18$ sebagai token. Apabila karakter berikutnya adalah `=`, maka lexer akan berpindah ke state $Q19$ untuk menghasilkan token (`>=`).

7. Penanganan Assignment dan Equality

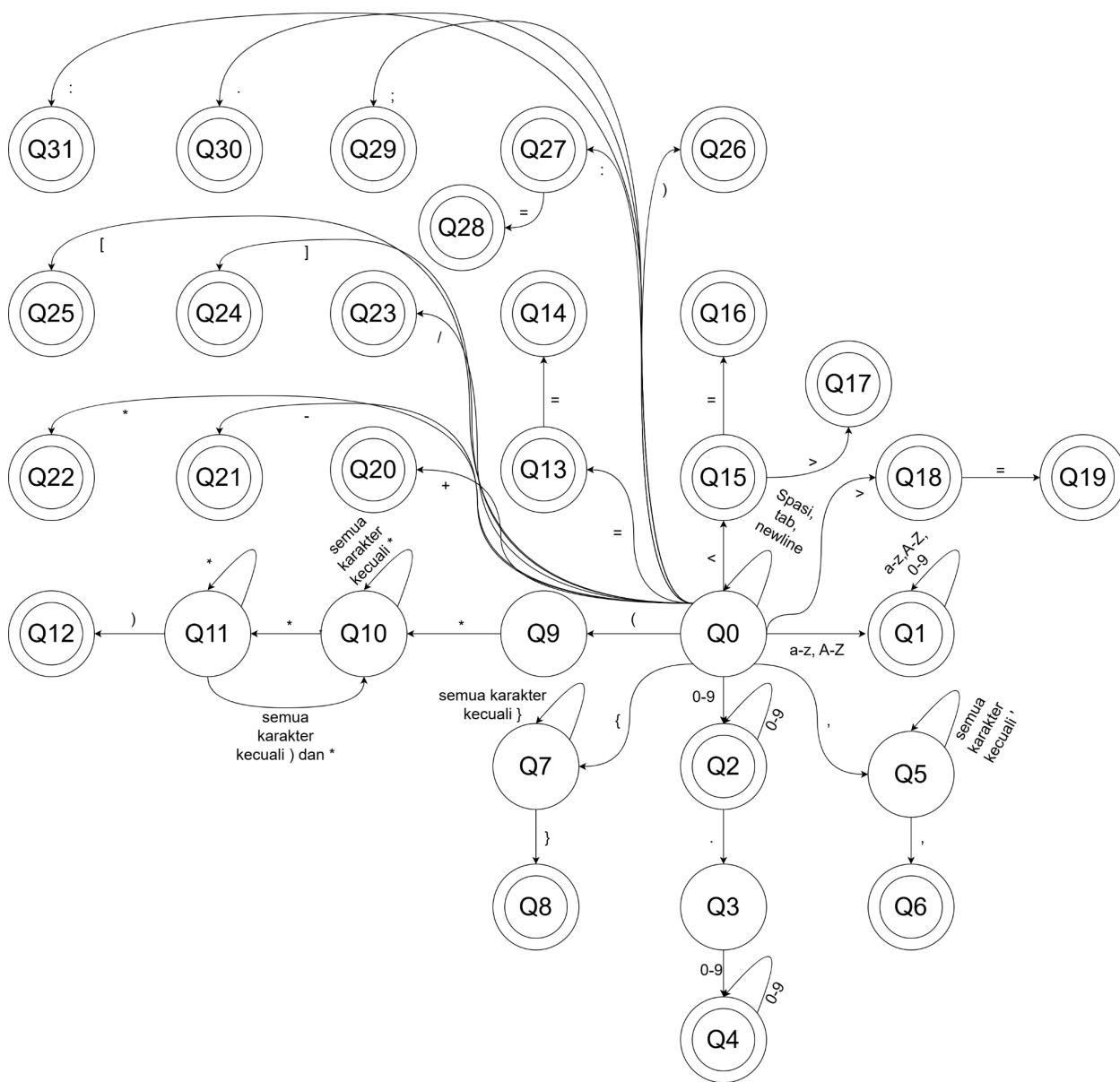
Selain itu, diagram ini juga membedakan jalur penanganan untuk operator assignment dan equality secara spesifik untuk mencegah terjadinya ambiguitas pembacaan. Untuk mengenali operator assignment (`:=`), lexer menggunakan jalur transisi yang diawali dengan pembacaan karakter titik dua (`:`) menuju state $Q27$. Transisi kemudian dilanjutkan ke state $Q28$ apabila karakter berikutnya adalah sama dengan (`=`), yang pada akhirnya menghasilkan token. Sementara itu, operator equality ditangani melalui jalur yang berbeda. Pembacaan karakter sama dengan (`=`) pertama akan membawa lexer ke state $Q13$. Jika karakter `=` tersebut langsung diikuti oleh karakter `=` kedua, transisi akan berlanjut ke state $Q14$ untuk mengidentifikasi dan menghasilkan token (`==`).

8. Penanganan Tanda Baca dan Operator Aritmatika

Diagram ini langsung dari Q0 menuju final state individu untuk operator dan simbol

berkarakter tunggal, antara lain:

- + menuju Q20
- - menuju Q21
- * menuju Q22
- / menuju Q23
-] menuju Q24
- [menuju Q25
-) menuju Q26
- ; menuju Q29
- , menuju Q30
- . menuju Q31



2. Implementasi Program

2.1. token.hpp

File ini adalah "kamus" dari seluruh program. Isinya dua hal: pertama, enum class TokenType yang mendaftarkan semua 52 jenis token yang dikenali bahasa Arion — mulai dari literal seperti INTCON dan REALCON, operator seperti PLUS dan BECOMES, keyword seperti PROGRAMSY dan BEGINSY, hingga IDENT,

COMMENT, dan ERROR_TOK. Kedua, struct Token yang menjadi "wadah" untuk satu token hasil lexing, menyimpan tiga informasi: jenis tokennya (type), nilainya (value), dan di baris berapa token itu ditemukan (line).

2.2. token.cpp

File ini mengimplementasikan dua fungsi pendukung dari token.hpp. Pertama, tokenTypeName() yang mengubah TokenType menjadi string — misalnya TokenType::PLUSY menjadi "plus" — digunakan saat program mencetak output ke terminal maupun file. Kedua, tokenHasValue() yang menentukan apakah suatu token perlu mencetak value-nya atau tidak — misalnya ident (Hello) perlu value, sedangkan semicolon tidak.

2.3. identifier.hpp

File ini adalah deklarasi (header) dari modul pengenalan identifier dan keyword. Isinya hanya dua baris deklarasi fungsi: isIdentChar() dan classifyIdent(). File ini di-include oleh siapapun yang perlu menggunakan kedua fungsi tersebut.

2.4. identifier.cpp

File ini adalah implementasi dari kedua fungsi yang dideklarasikan di Identifier.hpp. Fungsi isIdentChar() mengecek apakah sebuah karakter boleh menjadi bagian dari identifier, yaitu harus berupa huruf atau digit. Fungsi classifyIdent() menerima sebuah kata dalam bentuk lowercase, lalu mencocokkannya satu per satu dengan seluruh keyword yang ada di bahasa Arion — seperti "program", "begin", "while", "div", "not", dan lainnya. Jika cocok, dikembalikan TokenType keyword yang sesuai. Jika tidak cocok dengan keyword manapun, dikembalikan TokenType::IDENT yang berarti kata tersebut adalah identifier biasa seperti nama variabel atau nama fungsi. Karena Arion bersifat case-insensitive, konversi ke lowercase dilakukan oleh pemanggil sebelum fungsi ini dipanggil, sementara value asli identifier tetap disimpan dalam bentuk original-nya.

2.5. literal.hpp

File ini berisi deklarasi modul yang bertugas untuk mengenali serta memproses token bertipe literal. Dalam bahasa Arion, literal mencakup nilai numerik seperti konstanta integer dan real, serta data teks berupa karakter tunggal maupun string. Pada file ini didefinisikan dua fungsi utama, yaitu `handleNumber()` untuk memproses pembacaan angka, dan `handleStringOrChar()` untuk memproses karakter maupun kumpulan karakter.

2.6. literal.cpp

File ini berisi implementasi dari fungsi-fungsi pemindaian literal yang sebelumnya telah dideklarasikan. Secara umum, terdapat dua proses utama di file ini.

Fungsi `handleNumber()` digunakan untuk membaca serta mengklasifikasikan literal numerik. Proses dimulai dari digit awal, kemudian dilanjutkan dengan pembacaan karakter berikutnya. Jika ditemukan tanda titik, dilakukan pengecekan terhadap karakter setelahnya untuk menentukan apakah bilangan tersebut termasuk bilangan riil atau tetap sebagai bilangan integer. Apabila setelah titik terdapat digit, maka dikategorikan sebagai bilangan riil, sedangkan jika tidak, proses dihentikan dan diklasifikasikan sebagai bilangan integer.

Fungsi `handleStringOrChar()` digunakan untuk menangani literal teks yang diawali dengan tanda petik tunggal. Karakter akan dibaca hingga ditemukan tanda penutup, kemudian hasilnya diklasifikasikan berdasarkan panjangnya. Jika hanya terdiri dari satu karakter, maka dianggap sebagai karakter tunggal, sedangkan jika lebih dari satu karakter, maka dikategorikan sebagai string.

2.7. operator.hpp

File ini merupakan file deklarasi untuk modul yang bertanggung jawab menangani pemindaian operator dan tanda baca pada bahasa Arion. File ini mendaftarkan dua fungsi utama yang dapat dipanggil oleh bagian program lain, yaitu fungsi `isOperatorChar()` dan `classifyOperator()`.

2.8. operator.cpp

File ini berisi implementasi dari fungsi-fungsi yang telah dideklarasikan sebelumnya untuk menangani pengenalan simbol. Secara umum, terdapat dua peran utama yang dijalankan, yaitu melakukan pengecekan awal terhadap karakter serta mengklasifikasikan simbol yang telah dikenali. Fungsi `isOperatorChar()` digunakan untuk memverifikasi apakah suatu karakter tunggal termasuk dalam kategori karakter yang dapat membentuk operator atau tanda baca. Setelah karakter dipastikan valid, fungsi `classifyOperator()` akan memproses untaian karakter tersebut dan memetakannya ke dalam jenis token yang sesuai. Proses klasifikasi ini mencakup beberapa kategori, antara lain operator aritmatika, operator relasional, serta tanda baca yang berfungsi mengatur struktur penulisan program seperti kurung, koma, titik koma, dan titik.

2.9. lexer.hpp

File ini merupakan kerangka dari kelas `lexer`. Pada file ini, didefinisikan kelas `Lexer` yang menyimpan variabel `inputStream` untuk membaca file, `currentChar` untuk melacak karakter aktif, dan `currentLine` untuk menghitung baris. Selain itu, file ini menyediakan fungsi yang dapat dipanggil oleh `main.cpp` untuk menyelesaikan lexical analysis.

2.10. lexer.cpp

File ini berisi implementasi dari kelas `lexer` yang terdapat pada `lexer.hpp`. Terdapat beberapa fungsi yang dibuat implementasinya pada file ini, yaitu: `readNextChar()`, `skipWhitespace()`, `getNextToken()`, dan `isEOF()`. Fungsi tersebut memiliki peran yang sangat penting dalam program ini. `getNextToken()` merupakan fungsi utama yang ada di kelas `lexer`. Fungsi tersebut ditujukan untuk membaca setiap karakter secara satu persatu dan kemudian menentukan output dari token yang ada menggunakan prinsip DFA. Fungsi `skipWhitespace()`, `readNextChar()`, dan `isEOF()` juga digunakan di dalam fungsi `getNextToken()` untuk membantu menentukan arah penyelesaian dari setiap kemungkinan yang ada.

2.11. main.cpp

File ini bertugas untuk menangani alur input dan output dasar, dimulai dari menerima argumen command line berupa nama file source code yang akan diproses. Program kemudian akan melakukan validasi file dan membuka file tersebut. Jika file berhasil dibuka, program akan membaca isi kode sumber secara sekuensial. .

BAB III

PENGUJIAN

1. Test Case 1

1.1. Input :

```
Masukkan alamat file.txt: /home/nswaea/Documents/Tubes/TBFO/KOI-Tubes-IF2224-2026/test/1.txt

Berhasil membuka file: /home/nswaea/Documents/Tubes/TBFO/KOI-Tubes-IF2224-2026/test/1.txt
--- Isi File Input ---
program Hello;
var
a, b: integer;
begin
a := 5;
b := a + 10;
writeln('Result = ', b);
end.
```

1.2. Output:

```
--- Memulai proses baca karakter ---
programsy
ident (Hello)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon
ident (writeln)
lparent
string ('Result = ')
comma
ident (b)
rparent
semicolon
endsy
period

--- Selesai ---
Output berhasil disimpan di: /home/neswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/1_solusi.txt
```

2. Test Case 2

2.1. Input:

```
Masukkan alamat file.txt: /home/neswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/2.txt
Berhasil membuka file: /home/neswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/2.txt
--- Isi File Input ---
a := 10;
if a <> 5 then
  b <= 20;
if a >= 15 then
  c == 30;
```

2.2. Output:


```
--- Memulai proses baca karakter ---
ident (a)
becomes
intcon (10)
semicolon
ifsy
ident (a)
neq
intcon (5)
thensy
ident (b)
leq
intcon (20)
semicolon
ifsy
ident (a)
geq
intcon (15)
thensy
ident (c)
eq1
eq1
intcon (30)
semicolon

--- Selesai ---
Output berhasil disimpan di: /home/nswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/2_solusi.txt
```

3. Test Case 3

3.1. Input:

```
Masukkan alamat file.txt: /home/nswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/3.txt
Berhasil membuka file: /home/nswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/3.txt
--- Isi File Input ---
PROGRAM MyTest;
VAR
  X: INTEGER;
BEGIN
  If X > 0 Then
    X := X DIV 2;
END.
```

3.2. Output:

```
--- Memulai proses baca karakter ---
ident (PROGRAM)
ident (MyTest)
semicolon
ident (VAR)
ident (X)
colon
ident (INTEGER)
semicolon
ident (BEGIN)
ident (If)
ident (X)
gtr
intcon (0)
ident (Then)
ident (X)
becomes
ident (X)
ident (DIV)
intcon (2)
semicolon
ident (END)
period

--- Selesai ---
Output berhasil disimpan di: /home/nswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/3_solusi.txt
```

4. Test Case 4

4.1. Input:

```
Masukkan alamat file.txt: /home/nswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/4.txt

Berhasil membuka file: /home/nswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/4.txt
--- Isi File Input ---
array[1..10]of integer;
procedure(a,b:char);
begin(x);end.
```

4.2. Output:

```
--- Memulai proses baca karakter ---
arraysy
lbrack
intcon (1.)
period
intcon (10)
rbrack
ofsy
ident (integer)
semicolon
proceduresy
lparent
ident (a)
comma
ident (b)
colon
ident (char)
rparent
semicolon
beginsy
lparent
ident (x)
rparent
semicolon
endsy
period

--- Selesai ---
Output berhasil disimpan di: /home/neswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/4_solusi.txt
```

5. Test Case 5

5.1. Input:

```
Masukkan alamat file.txt: /home/neswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/5.txt

Berhasil membuka file: /home/neswaea/Documents/Tubes/TBF0/KOI-Tubes-IF2224-2026/test/5.txt
--- Isi File Input ---
const
  Initial = 'A';
  Message = 'Ini testcase';
  Empty = '';
```

5.2. Output:



```
--- Memulai proses baca karakter ---
constsy
ident (Initial)
eql
charcon ('A')
semicolon
ident (Message)
eql
string ('Ini testcase')
semicolon
ident (Empty)
eql
string ('')
semicolon

--- Selesai ---
Output berhasil disimpan di: /home/neswaea/Documents/Tubes/TBF0/K0I-Tubes-IF2224-2026/test/5_solusi.txt
```

BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Selain itu, implementasi program juga telah menerapkan struktur modular dengan memisahkan penanganan identifier, keyword, literal, operator, simbol, dan representasi token ke dalam modul yang berbeda. Pendekatan ini mempermudah pengembangan serta meningkatkan keterbacaan dan maintainability kode.

Dari hasil perancangan, implementasi, dan pengujian yang telah dilakukan, didapatkan kesimpulan bahwa program telah berhasil dibuat dan dikembangkan sesuai dengan spesifikasi yang diberikan. Program telah mampu membaca file .txt dan mengubahnya menjadi daftar token yang merepresentasikan unit-unit bermakna dalam bahasa arion. Proses ini dilakukan dengan menggunakan pendekatan **Deterministic Finite Automata (DFA)**, di mana setiap karakter dari source code yang telah dibuat akan dibaca secara berurutan dan diproses sesuai dengan aturan transisi yang telah dirancang.

Dari sisi fungsionalitas, program ini juga berhasil mengenali berbagai jenis token, antara lain :

1. Identifier dan keyword
2. Literal (integer, real, string, dan karakter)
3. Operator aritmatika, relasional, dan assignment
4. Simbol dan tanda baca

Selain itu, struktur program juga telah dibentuk secara modular sesuai dengan penanganan yang ingin dilakukan, yaitu penanganan identifier, literal, operator, dan representasi token ke dalam modul yang berbeda. Tujuannya adalah mempermudah pengembangan, maintainability code, dan juga meningkatkan kemudahan dalam membaca kode.

Hasil Pengujian memperlihatkan bahwa program berhasil mengubah source code mentah menjadi daftar token sesuai dengan format yang ditentukan dengan baik. Dengan demikian, dapat

disimpulkan bahwa program telah memenuhi tujuan utama dari milestone pertama, yaitu membangun lexical analyzer yang bekerja sesuai prinsip DFA.

2. Saran

Meskipun program telah berjalan dengan baik, masih terdapat beberapa aspek yang bisa dikembangkan untuk meningkatkan kualitas dan fungsionalitas sistem. Pertama, pada file input dengan ukuran yang besar, Lexical Analyzer akan tidak efisien karena akan sangat lama untuk melakukan translasi dari source code menjadi tokennya. Oleh karena itu, dapat dilakukan optimasi pada implementasi DFA, salah satu pendekatan yang dapat digunakan adalah memperjelas struktur state dan transisi dalam bentuk tabel atau diagram yang lebih sistematis.

BAB V

LAMPIRAN

1. Link Repository GitHub

Berikut adalah tautan untuk mengakses repository pada platform GitHub:

<https://github.com/Alvacodee/KOI-Tubes-IF2224-2026>

2. Link Workspace Diagram Transisi DFA

Berikut adalah tautan untuk mengakses workspace pada platform draw.io:

<https://drive.google.com/file/d/1RiAAAnoB2hjkNPZm29Pn6T1FA5qK6AiLD/view?usp=sharing>

3. Pembagian Tugas

NAMA	NIM	PEMBAGIAN TUGAS
Zahran Alvan Putra Winarko	13524124	Handle identifier & keyword (variabel, nama, program, var, dll)
Neswa Eka Anggara	13524136	Handle comment, whitespace & gabung semua (flow lexer/DFA)
Daniel Putra Rywandi S.	13524143	Handle angka & literal (int, real, char, string) , membuat dfa



Muh. Hartawan Haidir	13524147	Handle operator & simbol (+, :=, <=, ;, dll)
----------------------	----------	--

BAB VI

REFERENSI

1. Sekawan Media. (n.d.). *DFD adalah: Pengertian, Fungsi, dan Contohnya*. Diakses pada 2 April 2026, dari <https://www.sekawanmedia.co.id/blog/dfd-adalah/>
2. IBM. (n.d.). *Data Flow Diagram (DFD)*. Diakses pada 2 April 2026, dari <https://www.ibm.com/id-id/think/topics/data-flow-diagram>
3. Binus University. (2024). *Perbedaan DFD dan UML dalam Pemodelan Sistem*. Diakses pada 2 April 2026, dari <https://binus.ac.id/bekasi/2024/11/perbedaan-dfd-dan-uml-dalam-pemodelan-sistem/>
4. Trivusi. (2022). *Finite State Automata: Pengertian dan Penjelasan*. Diakses pada 2 April 2026, dari <https://www.trivusi.web.id/2022/08/finite-state-automata.html>
5. Edunex ITB. (n.d.). *Formal Language and Automata Theory Materials*. Diakses pada 2 April 2026, dari <https://edunex.itb.ac.id/>